

Uniwersytet Wrocławski
Wydział Fizyki i Astronomii

Oliwer Urbaniak

System zarządzania pracą drukarki 3D w laboratorium
studenckim.

Praca inżynierska na kierunku
Informatyka Stosowana i Systemy Pomiarowe

Opiekun
dr Bartosz Brzostowski

Wrocław, 17 maja 2026

Spis treści

1	Wprowadzenie	6
1.1	Motywacja i cel pracy	6
1.2	Zakres pracy	6
1.3	Struktura pracy	6
2	Analiza wymagań i przegląd rozwiązań	7
2.1	Wymagania funkcjonalne	7
2.2	Wymagania niefunkcjonalne	8
2.3	Przegląd istniejących rozwiązań	8
2.3.1	Octoprint	8
2.3.2	Mainsail i Fluidt	8
2.3.3	Rozwiązania komercyjne	8
2.3.4	Wnioski	8
3	Wybór technologii	9
3.1	Języki programowania	9
3.1.1	TypeScript / Node.js	9
3.1.2	Python	9
3.2	Framework webowy - Express.js	9
3.3	Baza danych - Azure Cosmos DB	9
3.4	Przechowywanie plików - Azure Blob Storage	9
3.5	Kolejkowanie - Azure Service Bus	9
3.6	Sterowanie urządzeniem - Azure IoT Hub	9
3.7	Frontend - React i Vite	9
3.8	Kontenery - Docker i Docker Compose	9
3.9	CAD - SolidWorks	9
3.10	Porównanie wybranych alternatyw	9
4	Architektura systemu	10
4.1	Ogólna struktura	10
4.2	Diagram architektury	10
4.3	Model danych	10
4.3.1	Użytkownik (kontener <code>users</code>)	10
4.3.2	Zadanie (kontener <code>jobs</code>)	10
4.3.3	Refresh Token (kontener <code>refresh-token</code>)	10
4.4	Cykl życia zadania do wydruku	10
4.5	Bezpieczeństwo i uwierzytelnianie	10
5	Implementacja serwisów backendowych	11
5.1	Auth Service	11
5.1.1	Opis ogólny	11
5.1.2	Endpointy API	11
5.1.3	Kluczowe fragmenty implementacji	11
5.1.4	Weryfikacja e-mail	11
5.2	User Service	11
5.2.1	Opis ogólny	11

5.2.2	Endpointy API	11
5.2.3	Mechanizm autoryzacji	11
5.3	Files Service	11
5.3.1	Opis ogólny	11
5.3.2	Przepływ przesyłania pliku	11
5.4	Queue Service	11
5.4.1	Opis ogólny	11
5.4.2	Listener kolejki	11
5.4.3	Endpointy HTTP API	11
5.5	Printer Service	11
5.5.1	Opis ogólny	11
5.5.2	Harmonogram	11
5.5.3	Monitor gotowości	11
5.5.4	Listener telemetrii	11
5.5.5	Endpointy HTTP API	11
5.6	Video Service	11
6	Implementacja frontendu	12
6.1	Struktura aplikacji	12
6.2	Klient API	12
6.3	Routing i ochrona tras	12
6.4	Ekrany aplikacji	12
6.4.1	Strona główna	12
6.4.2	Dashboard użytkownika	12
6.4.3	Upload i planowanie	12
6.4.4	Kontrola druku	12
6.4.5	Panel administracyjny	12
6.5	Internacjonalizacja	12
6.6	Obsługa motywu	12
7	Agent Raspberry Pi - AddiPi Agent	13
7.1	Opis ogólny	13
7.2	Architektura agenta	13
7.3	Połączenie z IoT Hub	13
7.4	Obsługiwane metody bezpośrednie	13
7.5	Przebieg obsługi metody startPrint	13
7.6	Telemetria	13
7.7	Komunikacja z OctoPrint	13
8	Obudowa urządzenia - projekt CAD	14
8.1	Cel i zakres projektu mechanicznego	14
8.2	Moduł CASE - obudowa Raspberry Pi z kamerą	14
8.2.1	Podstawa projektowa	14
8.2.2	Zakres modyfikacji	14
8.2.3	Mocowanie kamery	14
8.2.4	Struktura plików CASE	14
8.3	Moduł STAND - podstawka regulowana	14
8.4	Złożenie całości	14

8.5	Parametry wydruku	14
8.6	Instrukcja montażu	14
9	Konfiguracja Raspberry Pi	15
9.1	Opis środowiska	15
9.2	Instalacja agenta	15
9.3	Automatyczne uruchamianie agenta przez systemd	15
9.4	Tunel Cloudflare - ekspozycja kamery	15
9.4.1	Instalacja cloudflared	15
9.4.2	Skrypt publikowania URL kamery	15
9.4.3	Cykliczne odnawianie URL	15
9.5	Konfiguracja crontab	15
9.5.1	Uzasadnienie parametrów	15
9.6	Problem z dostępnością sieci przy starcie	15
9.7	Podsumowanie konfiguracji Raspberry Pi	15
10	Wdrożenie i infrastruktura	16
10.1	Infrastruktura chmurowa	16
10.2	Inicjalizacja infrastruktury	16
10.3	Konteneryzacja - Dockerfiles	16
10.4	Docker Compose	16
10.5	Zmienne środowiskowe	16
10.6	Wdrożenie frontendu	16
10.7	Wdrożenie agenta	16
11	Testowanie systemu	17
11.1	Metodologia testowania	17
11.2	Testy jednostkowe - Auth Service	17
11.3	Testy integracyjne przez Postman	17
11.4	Test end-to-end - pełny przepływ druku	17
11.5	Testy wydajnościowe i obserwacje	17
12	Podsumowanie	18
12.1	Osiągnięcia	18
12.2	Napotkane trudności	18
12.3	Możliwości rozwoju	18
12.4	Wnioski	18
	Appendices	20
A	Konfiguracja środowiska deweloperskiego	20
A.1	Wymagania	20
A.2	Pierwsze uruchomienie	20
B	Struktura repozytoriów	20
C	Endpointy API - podsumowanie	20

Streszczenie

Niniejsza praca inżynierska opisuje projekt, implementację oraz wykonanie systemu *AddiPi* - rozproszonego systemu zarządzania pracą drukarki 3D przeznaczonego dla laboratorium studenckiego. System umożliwia użytkownikom przysyłanie plików G-code, planowanie oraz monitorowanie zadań do wydruku, a administratorom - pełną kontrolę nad kolejką i użytkownikami.

Architektura systemu oparta jest na podejściu mikroserwisowym. Wyodrębniono sześć niezależnych serwisów backendowych: uwierzytelniania (Auth Service), zarządzania użytkownikami (User Service), obsługi plików (Files Service), kolejkowania zadań (Queue Service), sterowania drukarką (Printer Service) oraz agenta działającego na urządzeniu Raspberry Pi (AddiPi Agent). Całość uzupełnia aplikacja frontendowa zbudowana w React oraz serwis wideo do podglądu kamery.

Komunikacja między serwisami odbywa się za pośrednictwem protokołu HTTP/REST oraz chmurowej kolejki Azure Service Bus. Dane przechowywane są w Azure Cosmos DB, a pliki G-code w Azure Blob Storage. Sterowanie drukarką realizowane jest przez Azure IoT Hub przy użyciu bezpośrednich wywołań metod (Direct Methods) na Raspberry Pi, na którym działa oprogramowane OctoPrint.

System zrealizowano przy użyciu języków TypeScript (Node.js) i Python. Wdrożenie odbywa się w kontenerach Docker, koordynowanych plikiem Docker Compose. Interfejs użytkownika obsługuje uwierzytelnianie tokenem JWT, przysyłanie plików, planowanie zadań, podgląd kamery w czasie rzeczywistym oraz panel administracyjny.

Słowa kluczowe: mikroserwisy, drukarka 3D, OctoPrint, Raspberry Pi, Azure Iot Hub, Azure Cosmos DB, Docker, TypeScript, Python, REST API, kolejkowanie zadań

1 Wprowadzenie

1.1 Motywacja i cel pracy

Drukarki 3D stanowią obecnie standardowe wyposażenie laboratoriów akademickich, warsztatów studenckich oraz zakładów produkcyjnych zorientowanych na wytwarzanie przyrostowe. Urządzenia te, pomimo rosnącej dostępności i malejących cen, pozostają zasobem współdzielonym - ich liczba jest zwykle ograniczona, a czas drukowania pojedynczego modelu może wynosić od kilkunastu minut do wielu godzin. W środowisku akademickim oraz przemysłowym rodzi to naturalne problemy organizacyjne: konieczność ręcznego umawiania się na czas druku, brak przejrzystości stanu kolejki oraz niemożność zdalnego monitorowania postępu pracy.

Celem niniejszej pracy jest zaprojektowanie i implementacja systemu *AddiPi*, który automatyzuje proces zarządzania drukarką 3D w laboratorium studenckim. System pozwala użytkownikom na:

- przesyłanie plików G-code przez przeglądarkę internetową,
- planowanie druków na określony termin lub dodawanie ich do kolejki,
- śledzenie postępu druku w czasie rzeczywistym, wraz z podglądem kamery,
- otrzymywanie powiadomień o zakończeniu lub błędzie druku.

Administratorzy laboratorium zyskują natomiast narzędzia do zarządzania użytkownikami, przeglądania historii wszystkich zadań oraz sterowania kolejką i urządzeniami.

1.2 Zakres pracy

Praca obejmuje:

1. analizę wymagań systemu i wybór technologii,
2. projekt architektury mikroservisowej,
3. implementację wszystkich komponentów backendowych i frontendowych,
4. konfigurację infrastruktury chmurowej (Microsoft Azure),
5. wdrożenie systemu przy użyciu kontenerów Docker,
6. testy poprawności działania.

1.3 Struktura pracy

Rozdział 2 zawiera analizę wymagań i przegląd istniejących rozwiązań. Rozdział 3 omawia wybrane technologie i uzasadnia decyzje projektowe. Rozdział 4 opisuje architekturę systemu. Rozdziały 5–7 szczegółowo omawiają implementację poszczególnych komponentów programowych. Rozdział 8 opisuje projekt mechaniczny obudowy w SolidWorks. Rozdział 9 omawia konfigurację Raspberry Pi, automatyzację przez cron i tunel Cloudflare. Rozdział 10 dotyczy wdrożenia i infrastruktury chmurowej Azure. Rozdział 11 przedstawia wyniki testów. Praca kończy się podsumowaniem w rozdziale 12.

2 Analiza wymagań i przegląd rozwiązań

2.1 Wymagania funkcjonalne

Na podstawie analizy potrzeb laboratorium studenckiego sformułowano następujące wymagania funkcjonalne:

Dla użytkowników:

- Rejestracja z weryfikacją adresu e-mail (ograniczoną do domeny uczelni @uwr.edu.pl).
- Logowanie z użyciem tokenów JWT (access token + refresh token).
- Przesyłanie plików G-code (maksymalnie 50 MB, format .gcode).
- Planowanie druku na wybrany termin lub dodanie do kolejki ogólnej.
- Przeglądanie własnych zadań z filtrowaniem według statusu.
- Podgląd postępu druku w czasie rzeczywistym (pasek postępu, temperatura dyszy i podłoża, obraz z kamery).
- Anulowanie i ponowne uruchamianie własnych zadań.

Dla administratorów:

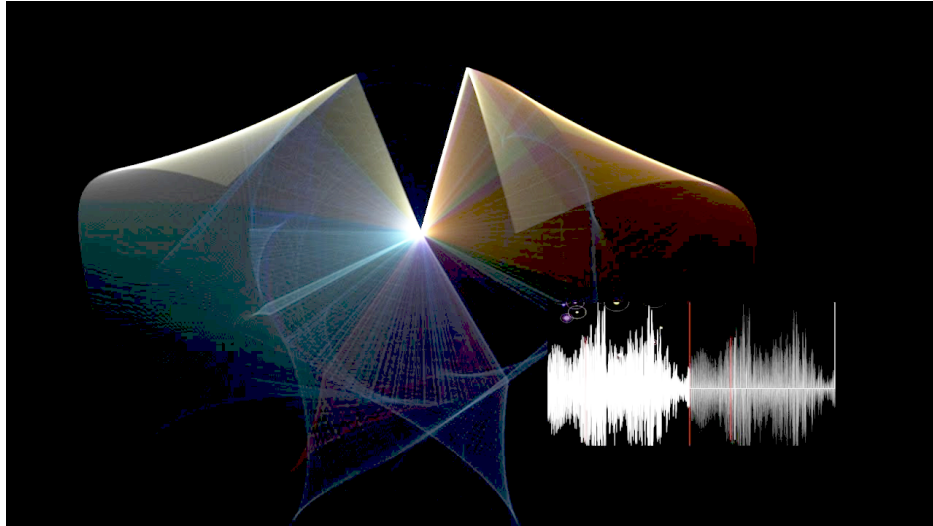
W swojej pracy możesz odnosić się do równań np. do równania (1) lub do rysunków, zarówno wektorowych, patrz rys. (1) jak i bitmapowych (2). Możesz też z powodzeniem cytować literaturę pojedynczo [2] lub kilka pozycji na raz [3, 1]. Jeśli chcesz to robić wygodniej załóż tzw. plik bibtex, ale możesz też pracować tak, jak w tym dokumencie.

$$\rho \left(\frac{\partial \vec{v}}{\partial t} + (\vec{v} \cdot \nabla) \vec{v} \right) = \rho \vec{f} - \nabla p + \mu \Delta \vec{v}, \quad (1)$$

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



Rysunek 1: Podpis rysunku, który jest obrazem wektorowym (EPS).



Rysunek 2: To jest rysunek drugi, kilkadziesiąt wahadeł podwójnych z syntezą dźwięku.

2.2 Wymagania нефunkcjonalne

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. .. in the figure 2.2.

2.3 Przegląd istniejących rozwiązań

2.3.1 Octoprint

2.3.2 Mainsail i Fluidt

2.3.3 Rozwiązania komercyjne

2.3.4 Wnioski

3 Wybór technologii

3.1 Języki programowania

3.1.1 TypeScript / Node.js

3.1.2 Python

3.2 Framework webowy - Express.js

3.3 Baza danych - Azure Cosmos DB

3.4 Przechowywanie plików - Azure Blob Storage

3.5 Kolejowanie - Azure Service Bus

3.6 Sterowanie urządzeniem - Azure IoT Hub

3.7 Frontend - React i Vite

3.8 Kontenery - Docker i Docker Compose

3.9 CAD - SolidWorks

3.10 Porównanie wybranych alternatyw

4 Architektura systemu

4.1 Ogólna struktura

4.2 Diagram architektury

4.3 Model danych

4.3.1 Użytkownik (kontener users)

4.3.2 Zadanie (kontener jobs)

4.3.3 Refresh Token (kontener refresh-token)

4.4 Cykl życia zadania do wydruku

4.5 Bezpieczeństwo i uwierzytelnianie

5 Implementacja serwisów backendowych

5.1 Auth Service

5.1.1 Opis ogólny

5.1.2 Endpointy API

5.1.3 Kluczowe fragmenty implementacji

5.1.4 Weryfikacja e-mail

5.2 User Service

5.2.1 Opis ogólny

5.2.2 Endpointy API

5.2.3 Mechanizm autoryzacji

5.3 Files Service

5.3.1 Opis ogólny

5.3.2 Przepływ przesyłania pliku

5.4 Queue Service

5.4.1 Opis ogólny

5.4.2 Listener kolejki

5.4.3 Endpointy HTTP API

5.5 Printer Service

5.5.1 Opis ogólny

5.5.2 Harmonogram

5.5.3 Monitor gotowości

5.5.4 Listener telemetrii

5.5.5 Enpointy HTTP API

5.6 Video Service

6 Implementacja frontendu

6.1 Struktura aplikacji

6.2 Klient API

6.3 Routing i ochrona tras

6.4 Ekrany aplikacji

6.4.1 Strona główna

6.4.2 Dashboard użytkownika

6.4.3 Upload i planowanie

6.4.4 Kontrola druku

6.4.5 Panel administracyjny

6.5 Internacjonalizacja

6.6 Obsługa motywu

7 Agent Raspberry Pi - AddiPi Agent

7.1 Opis ogólny

7.2 Architektura agenta

7.3 Połączenie z IoT Hub

7.4 Obsługiwane metody bezpośrednio

7.5 Przebieg obsługi metody startPrint

7.6 Telemetria

7.7 Komunikacja z OctoPrint

8 Obudowa urządzenia - projekt CAD

8.1 Cel i zakres projektu mechanicznego

8.2 Moduł CASE - obudowa Raspberry Pi z kamerą

8.2.1 Podstawa projektowa

8.2.2 Zakres modyfikacji

8.2.3 Mocowanie kamery

8.2.4 Struktura plików CASE

8.3 Moduł STAND - podstawka regulowana

8.4 Złożenie całości

8.5 Parametry wydruku

8.6 Instrukcja montażu

9 Konfiguracja Raspberry Pi

9.1 Opis środowiska

9.2 Instalacja agenta

9.3 Automatyczne uruchamianie agenta przez systemd

9.4 Tunel Cloudflare - ekspozycja kamery

9.4.1 Instalacja cloudflared

9.4.2 Skrypt publikowania URL kamery

9.4.3 Cykliczne odnawianie URL

9.5 Konfiguracja crontab

9.5.1 Uzasadnienie parametrów

9.6 Problem z dostępnością sieci przy starcie

9.7 Podsumowanie konfiguracji Raspberry Pi

10 Wdrożenie i infrastruktura

10.1 Infrastruktura chmurowa

10.2 Inicjalizacja infrastruktury

10.3 Konteneryzacja - Dockerfiles

10.4 Docker Compose

10.5 Zmienne środowiskowe

10.6 Wdrożenie frontendu

10.7 Wdrożenie agenta

11 Testowanie systemu

11.1 Metodologia testowania

11.2 Testy jednostkowe - Auth Service

11.3 Testy integracyjne przez Postman

11.4 Test end-to-end - pełny przepływ druku

11.5 Testy wydajnościowe i obserwacje

12 Podsumowanie

12.1 Osiągnięcia

12.2 Napotkane trudności

12.3 Możliwości rozwoju

12.4 Wnioski

Literatura

- [1] Alexandre Joel Chorin. Numerical solution of the navier-stokes equations. *Mathematics of computation*, 22(104):745–762, 1968.
- [2] Maciej Matyka, Arzhang Khalili, and Zbigniew Koza. Tortuosity-porosity relation in porous media flow. *Physical Review E*, 78(2):026306, 2008.
- [3] Dawid Strzelczyk and Maciej Matyka. Meshless lattice boltzmann method on random grids. *in preparation*, 2022.

Appendices

A Konfiguracja środowiska deweloperskiego

A.1 Wymagania

A.2 Pierwsze uruchomienie

B Struktura repozytoriów

C Endpointy API - podsumowanie